

Database Systems

数据库系统

Instructor: Yiwen Shen (沈轶闻), Ph.D.
Dept. of Software & Computer Engineering,
Ajou University, Korea
chrisshen@ajou.ac.kr

事务管理

Transactions Management

大纲

- 事务概念
- 事务状态
- 并发执行
- 可串行化
- 可恢复性
- 隔离措施的实施
- SQL 中的事务定义
- 测试可串行化性。

事务概念

- **事务** 是程序执行单元以用来**访问**并可能**更新**各种数据项。
- 例如，从账户 A 向账户 B 转账 50 元的交易：
 1. 读取(A)
 2. $A := A - 50$
 3. 写入(A)
 4. 读取(B)
 5. $B := B + 50$
 6. 写(B)
- 需要解决的两个主要问题：
 - 各种类型的**故障**，例如硬件故障和系统崩溃
 - 多个事务的**并发**执行

资金转移示例

将 50 美元从账户 A 转账到账户 B:

1. 读取(A)
2. $A := A - 50$
3. 写入(A)
4. 读取(B)
5. $B := B + 50$
6. 写(B)

- 原子性要求

- 如果交易在步骤 3 之后、步骤 6 之前失败，资金将会“丢失”，导致数据库状态不一致。
 - 故障可能是由软件或硬件引起的。
 - 系统应确保对部分执行的事务所做的更新不会反映在数据库中。

- 持久性要求 - 一旦用户收到交易完成的通知（即 50 元的转账已完成），即使发生软件或硬件故障，交易对数据库的更新也必须持续存在。

资金转账示例 (续)

- **一致性要求：**
 - 交易执行后，A 和 B 的总和保持不变。
- 一般来说，一致性要求包括
 - 明确指定的完整性约束，例如主键和外键
 - 隐式完整性约束
 - 例如，所有账户余额之和减去贷款金额之和必须等于手头现金的价值。
 - 交易必须基于一致的数据库。
 - 事务执行期间，数据库可能暂时出现不一致的情况。
 - 事务成功完成后，数据库必须保持一致。
 - 错误的交易逻辑会导致数据不一致。

资金转账示例 (续)

- **隔离要求** - 如果在步骤 3 和 6 之间, 允许另一个事务 **T2** 访问部分更新的数据库, 它将看到一个不一致的数据库 ($A + B$ 的总和将小于它应该的值)。

T1

1. 读取(A)
2. $A := A - 50$
3. 写(A)
4. 读取(B)
5. $B := B + 50$
6. 写(B)

T2

读取(A), 读取(B), 打印($A+B$)

- 如果**串行**运行事务, 可以轻松确保隔离特性。
 - 也就是说, 一个接一个地运行。
- 但是, 同时执行多个事务具有显著的优势, 我们稍后将会看到。

ACID 特性

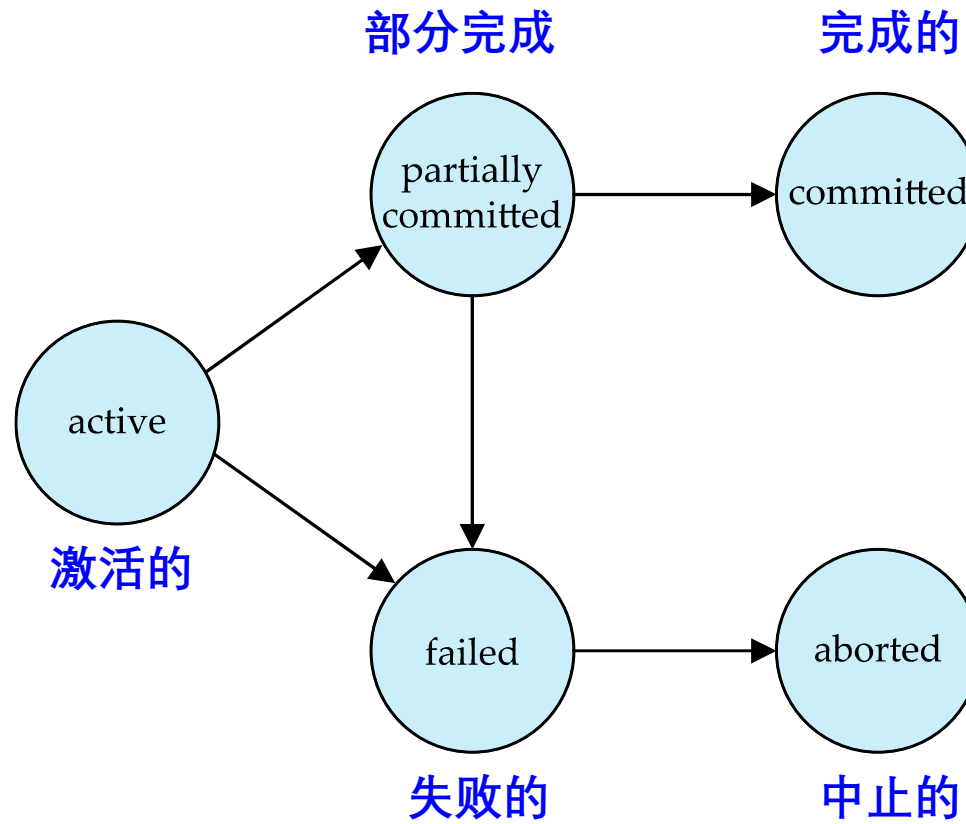
事务是程序执行的一个单元以访问并可能更新各种数据项。为了维护数据的完整性, 数据库系统必须确保:

- **原子性 (Atomicity)**.
 - 要么交易的所有操作都正确反映在数据库中, 要么一个操作都没有反映。
- **一致性 (Consistency)**.
 - 独立执行事务可以保持数据库的一致性。
- **隔离性 (Isolation)**.
 - 尽管可以同时执行多个事务, 但每个事务都必须对其他并发执行的事务一无所知。中间事务的结果必须对其他并发执行的事务隐藏。
 - 也就是说, 对于每一对交易 T_i 和 T_j , 要么 T_j 在 T_i 开始之前执行完毕, 要么 T_j 在 T_i 结束后开始执行。
- **耐用性 (Durability)**.
 - 事务成功完成后, 即使系统发生故障, 它对数据库所做的更改也会保留下来。

事务状态

- 激活的 (**active**) - 初始状态；事务在执行过程中保持此状态。
- 部分完成 (**partially committed**) - 在最终声明执行完毕之后。
- 失败的 (**failed**) - 在发现正常执行无法继续进行之后。
- 中止的 (**aborted**) - 在事务回滚并将数据库恢复到事务开始之前的状态之后。事务中止后有两种选择：
 - 重新启动事务
 - 只有在没有内部逻辑错误的情况下才能完成。
 - 终止事务
- 完成的 (**committed**) - 成功完成后。

事务状态 (续)



并发执行

- 系统允许**多个事务同时运行**。其优点包括：
 - 处理器和磁盘利用率提高，从而提升了事务**吞吐量 (Throughput)**。
 - 例如，一个事务可能正在使用 CPU，而另一个事务可能正在从磁盘读取数据或写入数据。
 - 平均响应时间缩短：短事务无需等待长事务。
- **并发控制方案 - 实现隔离的机制**
 - 也就是说，要控制并发事务之间的交互，以防止它们破坏数据库的一致性。

调度安排

- **调度 (schedule)** - 一系列指令，用于指定并发事务指令的执行顺序。
 - 一组事务的调度表必须包含这些事务的所有指令。
 - 必须保持指令在每笔事务中出现的顺序。
- 成功执行完毕的事务，其最后一条语句将是提交指令。
 - 默认情况下，事务假定执行提交指令作为其最后一步。
- 执行失败的事务将在最后一条语句中包含中止指令。

调度 1

- 设 T_1 从 A 向 B 转账 50 元, T_2 将余额的 10% 从 A 转账到 B 。
- T_1 是在之后紧接着 T_2 的串行调度:

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

调度2

- 串行调度方案中， T_2 之后是 T_1 。

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

调度3

- 设 T_1 和 T_2 为先前定义的事务。以下调度并非串行调度，但它是等价于调度1的。
- 在调度1、2 和 3 中， $A + B$ 之和保持不变。

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

调度4

- 右边的并行处理无法保存 $(A+B)$ 的值。

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

可串行化 (Serializability)

- 基本假设 - 每笔事务都能保持数据库的一致性。
- 因此，串行执行一组事务可以保持数据库的一致性。
- 如果一个（可能并发的）调度方案等价于一个串行调度方案，则称该调度方案是**可串行化**的。不同形式的调度方案等价性引出了以下概念：
 1. **冲突 可串行性 (Conflict serializability)**
 2. **视图 可串行化 (View serializability)**

事务的简化视图

- 我们忽略**除读写指令**之外的其他操作
- 我们假设事务可以在读写操作之间对本地缓冲区中的数据执行任意计算。
- 我们的简化计划只包含**读和写指令**。

相互矛盾的指令

- 事务 T_i 和 T_j 的指令 l_i 和 l_j ，是**冲突的**当且仅当存在某个数据条目 Q 被 l_i 和 l_j 访问，并且**至少有一条指令写入了 Q** 。
 - $l_i = \text{read}(Q), l_j = \text{read}(Q)$ 。 l_i 和 l_j 不冲突。
 - $l_i = \text{read}(Q), l_j = \text{write}(Q)$ 。它们冲突。
 - $l_i = \text{write}(Q), l_j = \text{read}(Q)$ 。它们冲突
 - $l_i = \text{write}(Q), l_j = \text{write}(Q)$ 。它们冲突。
- l_i 和 l_j 之间的**冲突**强制它们之间存在（逻辑上的）**时间顺序**。
- 如果 l_i 和 l_j 在一个调度表中是**连续的**，并且它们**不冲突**，即使它们在计划表中互换位置，它们的结果也保持不变。

调度3

T_1	T_2
read (A) $A := A - 50$ write (A)	read (A) $temp := A * 0.1$ $A := A - temp$ write (A)
read (B) $B := B + 50$ write (B) commit	read (B) $B := B + temp$ write (B) commit

冲突

不冲突

调度1

T_1 是在紧接着 T_2 之后的串行调度:

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

冲突可串行性 **Conflict serializability**

- 通过一系列**不冲突指令**的交换可以将调度 S 转换为调度 S' ，我们就说 S 和 S' 是 **冲突等价的 (conflict equivalent.)**。
- 如果调度 S 与 一个**串行调度** 是**冲突等价的**，我们称调度 S 是**冲突可串行化的**。
 - 例如：调度3是冲突可串行化的，因为它和串行调度1是冲突等价的。

冲突可串行化 (续)

- 通过一系列非冲突指令的交换，调度3 可以转换为 调度6，即 T_2 紧随 T_1 之后的串行调度方案。因此，**调度 3** 是冲突可串行化的。

T_1	T_2
read (A) write (A)	
	read (A) write (A)
read (B) write (B)	
	read (B) write (B)

调度3

T_1	T_2
read (A) write (A) read (B) write (B)	
	read (A) write (A) read (B) write (B)

调度6 (调度1)

冲突可串行化 (续)

- 不可冲突可串行化的 调度 示例:

T_3	T_4
read (Q)	write (Q)
write (Q)	

- 我们无法交换上述调度中的指令，以获得串行调度 $\langle T_3, T_4 \rangle$ 或串行调度 $\langle T_4, T_3 \rangle$ 。

视图可串行性 View Serializability

- 设 S 和 S' 为两个调度表，具有**相同的事务操作集合**。如果满足以下三个条件，则对于每个数据项 Q ，他们是**视图等价的**：
 1. 如果在调度 S 中，事务 T_i 读取 Q 的初始值，然后在调度 S' ，事务 T_i 也必须读取 Q 的初始值。
 2. 如果在调度 S 中，事务 T_i 执行 $\text{read}(Q)$ 操作，得到的值为事务 T_j 的 $\text{write}(Q)$ 操作产生的 (如有)，则在调度 S' 中，事务 T_i 也必须 $\text{read}(Q)$ 由以下方式产生的 Q 值：对事务 T_j 执行相同的 $\text{write}(Q)$ 操作。
 3. 在调度 S 中，在执行最终的 $\text{write}(Q)$ 操作的事务 (如果有)，也必须在调度 S' 中执行最终的写入 (Q) 操作。
- 可以看出，视图等效性也完全基于**读取和写入**操作。

视图可串行化 (续)

- 调度S，如果它在视图上等同于串行调度，那么就是 **视图可串行化的**。
- 每个 **冲突可串行化的** 调度也都是 **视图可串行化的**。
- 下面是一个**视图可串行化** 但不是 **冲突可串行化** 的调度表。

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		write (Q)

- 每个无法冲突可串行化的 视图可串行化调度 都有**盲写(blind writes)**。
 - **盲写(blind writes)** : 写操作之前没有读取操作。

可串行化的其他概念

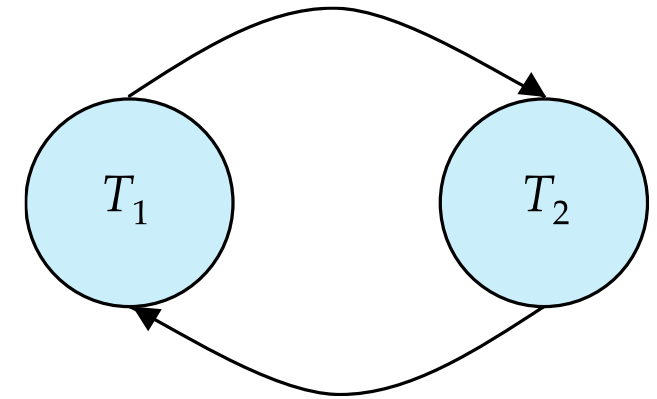
- 下面这个调度产生与串行调度 $\langle T_1, T_5 \rangle$ 同样的结果，但不是冲突或视图等价的。

T_1	T_5
read (A) $A := A - 50$ write (A)	
	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)

- 确定这种等效性需要分析读写以外的操作。

可串行化测试

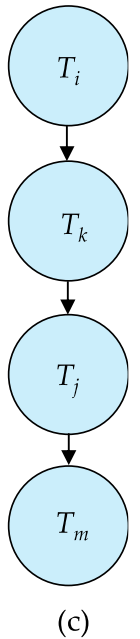
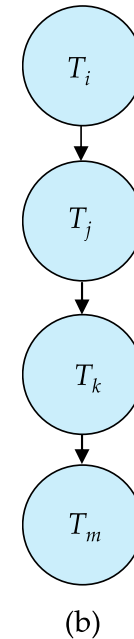
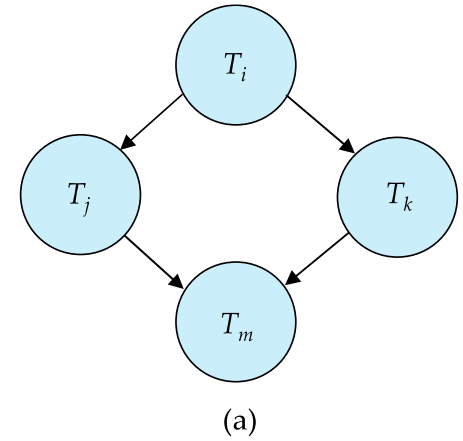
- 考虑一组事务 $T_1, T_2, \dots, T_n$ 的某个调度方案
- 优先关系图 (Precedence graph) - 一个有向图，其中节点是事务（名称）。
- 如果两个事务冲突，则从 T_i 至 T_j 画一条弧，且 T_i 访问了之前引发冲突的数据项。
- 我们可以用所访问的数据条目来标记弧。
- 例子：
 - 事务 T_i 和 T_j 的指令 l_i 和 l_j
 - 1. $l_i = \text{read}(Q), l_j = \text{write}(Q)$ 。它们冲突。
 - 2. $l_i = \text{write}(Q), l_j = \text{read}(Q)$ 。它们冲突
 - 3. $l_i = \text{write}(Q), l_j = \text{write}(Q)$ 。它们冲突。



T_1	T_2
read (A) $A := A - 50$	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B)
write (A) read (B) $B := B + 50$ write (B) commit	$B := B + temp$ write (B) commit

冲突可串行性测试

- 当且仅当调度的优先级图是**无循环**的时，该调度才是**冲突可串行化的**。
- **循环检测算法** 需用 n^2 的时间，其中 n 是图中的顶点数。
 - (更优的算法的复杂度为 $n + e$ ，其中 e 为边的数量。)
- 如果优先关系图是无循环的，对通过对该图进行 **拓扑排序 (topological sorting)** 来获得**可串行化顺序**。
 - 这是与图的偏序 (partial order) 一致的 线性序(linear order)。
 - 例如，A调度的序列化顺序为
 $T_5 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$
 - 还有其他的吗?



测试视图可串行化

- 冲突可串行化的优先级图测试不能直接用于测试视图可串行化。
 - 扩展以测试视图可串行化的成本随着优先级图的大小呈指数级增长。
- 检查调度是否可 视图串行化的问题 属于 NP 完全问题 (NP-complete)。
 - 因此，存在高效算法的可能性 极低。
- 然而，对于视图可序列化，一些实用的算法只需检查一些足够多的信息即可。
 - 但在数据库中实际使用的情况比较少见

可恢复的调度

需要解决事务失败时, 对并发的运行事务的影响。

- **可恢复调度** - 如果事务 T_j 读取了先前由事务 T_i 写入的数据项, 则 T_i 的提交操作出现在 T_j 的提交操作**之前**。
- 以下调度 (调度11) **不可恢复**

T_8	T_9
read (A) write (A)	
	read (A) commit
read (B)	

- 如果 T_8 应该中止, T_9 将会读取 (并可能向用户显示) 不一致的数据库状态。因此, 数据库必须确保调度是可恢复的。

级联回滚

- **级联回滚** - 单个事务的失败会导致一系列事务回滚。考虑以下调度，其中所有事务都尚未提交（因此该调度是可恢复的）。

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)	read (A) write (A)	
abort		read (A)

如果事务 T_{10} 失败，则事务 T_{11} 和 T_{12} 也 必须回滚。

- 可能导致大量工作付诸东流

无级联调度

- 无级联调度 - 级联回滚不会发生；
 - 对于每一对事务 T_i 并且 T_j 使得 T_j 读取先前由 T_i 写入的数据项，即 T_i 的提交操作出现在 T_j 的读取操作之前。
- 每个无级联调度程序也都是可恢复的。
- 最好将调度限制在非级联式的调度中。

并发控制

- 数据库必须提供一种机制，以确保**所有可能的调度**都能得到满足。
 - 要么**冲突**，要么 **视图可串行化**，并且
 - **可恢复**且最好是无级联的
- 一次只能执行一个事务的策略会生成串行调度，但并发性较差。
 - 串行调度是否可恢复/无级联？
- 在调度程序执行完毕后再测试其可串行化性就太晚了！
- **目标** - 开发能够保证可串行化的**并发控制协议**。

并发控制 (续)

- 为了保证数据库的**一致性**，调度必须能够避免冲突或视图序列化，并且是可恢复的，最好是无级联的。
- 一次只能执行一个事务的策略会生成串行调度，但并发性较差。
- 并发控制方案需要在允许的**并发量**和产生的**开销**之间进行**权衡**。
- 有些方案只允许生成 冲突可串行化 的调度，而另一些方案则允许生成视图可串行化但不可冲突串行化的调度。

并发控制 与 可串行化测试

- 并发控制协议允许并发调度，但要确保调度是冲突/视图可串行化的，并且是可恢复的和无级联的。
- 并发控制协议（通常）**不会在创建优先级图时**对其进行检查。
 - 相反，协议会施加一种约束，以避免不可串行化的调度。
- 不同的并发控制协议在允许的并发量和产生的开销之间提供了不同的权衡。
- 可串行化测试有助于我们理解为什么并发控制协议是正确的。

弱等级的一致性

- 有些应用程序愿意接受**较低的一致性水平**，允许使用不可串行化的调度方案。
 - 例如，一个只读事务，想要获取所有账户的大致总余额。
 - 例如，为优化查询而计算的数据库统计信息可能是近似值（为什么？）
 - 此类交易无需与其他交易进行序列化。
- 性能与精度之间的权衡

SQL-92 中的一致性级别

- **可序列化** - 默认
- **可重复读取** - 仅读取已提交的记录。
 - 重复读取同一条记录必须返回相同的值。
 - 但是，事务可能不具有可序列化性 - 它可能找到事务插入的一些记录，但找不到其他记录。
- **读取提交** - 只有已提交的记录才能读取。
 - 连续读取记录可能会返回不同的（但已提交的）值。
- **读取未提交** - 即使是未提交的记录也可以被读取。

一致性水平

- 较低程度的一致性有助于收集有关数据库的近似信息
- 警告：某些数据库系统默认情况下不保证调度程序的可序列化。
- 例如，Oracle（以及 9 版本之前的 PostgreSQL）默认支持一种称为快照隔离的一致性级别（并非 SQL 标准的一部分）。

SQL 中的事务定义

- 在 SQL 中，事务是隐式开始的。
- SQL 事务以以下方式结束：
 - 提交工作会提交当前事务并开始一个新的事务。
 - 回滚操作会导致当前事务中止。
- 在几乎所有数据库系统中，默认情况下，每个 SQL 语句如果成功执行，都会隐式地提交更改。
 - 可以通过数据库指令关闭隐式提交。
 - 例如，在 JDBC 中 `-- connection.setAutoCommit (false);`
- 隔离级别可以在数据库级别设置。
- 隔离级别可以在事务开始时更改。
 - 例如，在 SQL 中设置事务隔离级别为可序列化
 - 例如，在 JDBC 中 `-- connection.setTransactionIsolation (Connection.TRANSACTION_SERIALIZABLE)`

隔离级别的实施

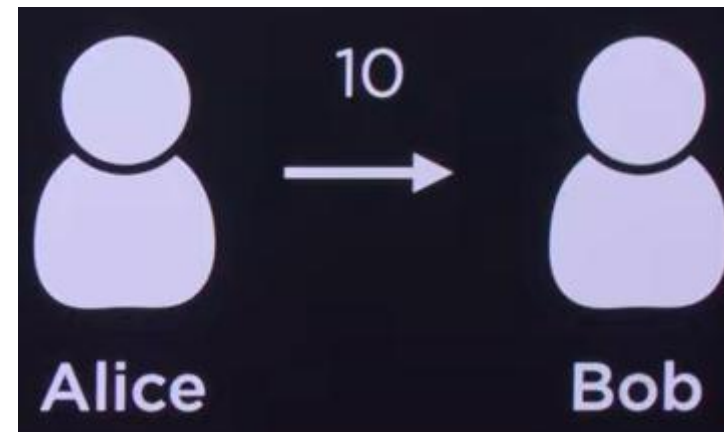
- 锁定
 - 锁定整个数据库与锁定单个项目
 - 按住锁定键多久？
 - 共享锁与独占锁
- 时间戳
 - 交易时间戳已分配，例如交易开始时的时间戳
 - 数据项存储两个时间戳
 - 读取时间戳
 - 写入时间戳
 - 时间戳用于检测乱序访问。
- 每个数据项有多个版本
 - 允许事务从数据库的“快照”中读取数据。

事务作为 SQL 语句

- 例如，事务1 (T1):
`select ID, name from instructor where salary > 90000`
- 例如，事务2 (T2) :
`insert into instructor values ('11111', 'James', 'Marketing', 100000)`
- 假设：
 - T1 开始，使用索引查找薪资大于 90000 的元组并锁定它们。
 - 然后T2执行。
 - T1 和 T2 是否存在冲突？元组级别的锁定能否检测到这种冲突？
 - 幻影现象的实例
- 另请考虑下面的 T3，其中 Wu 的薪资为 90000，
`update instructor
set salary = salary * 1.1
where name = 'Wu'`
- 核心思想：检测“谓词”冲突，并使用某种形式的“谓词锁定”。

事务示例

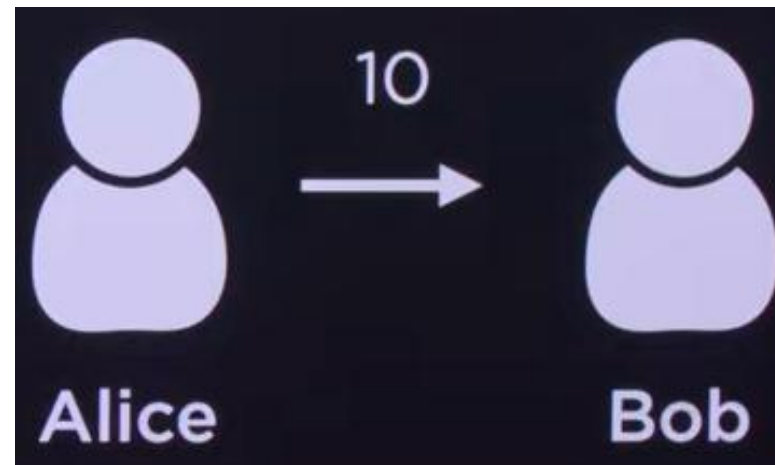
accounts		
id	name	balance
1	Alice	10
2	Bob	20
3	Charlie	30



事务示例

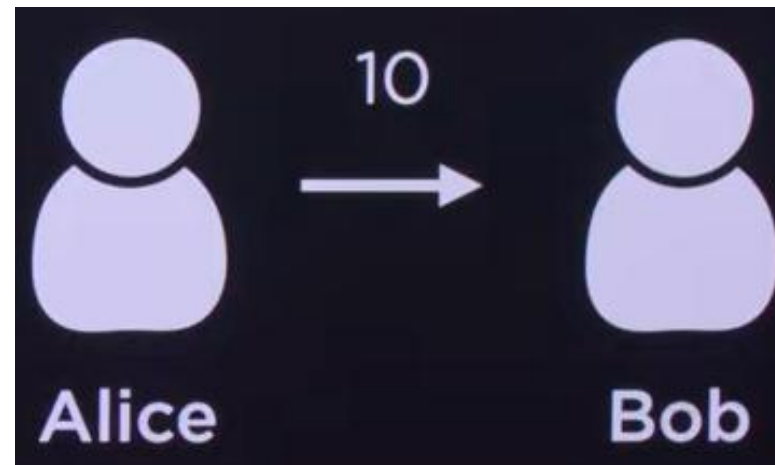
- 错误的逻辑

id	name	balance
1	Alice	10
2	Bob	30
3	Charlie	30



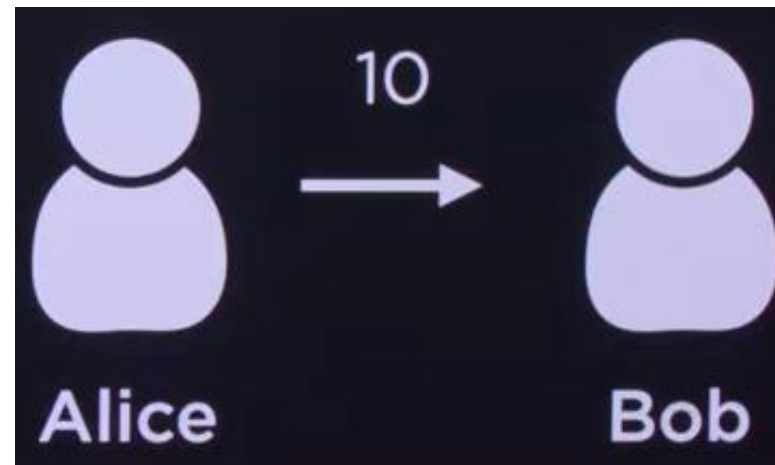
事务示例

id	name	balance
1	Alice	0
2	Bob	30
3	Charlie	30



事务示例

accounts		
id	name	balance
1	Alice	0
2	Bob	30
3	Charlie	30



事务 - 提交

BEGIN TRANSACTION;

...

COMMIT;

BEGIN TRANSACTION;

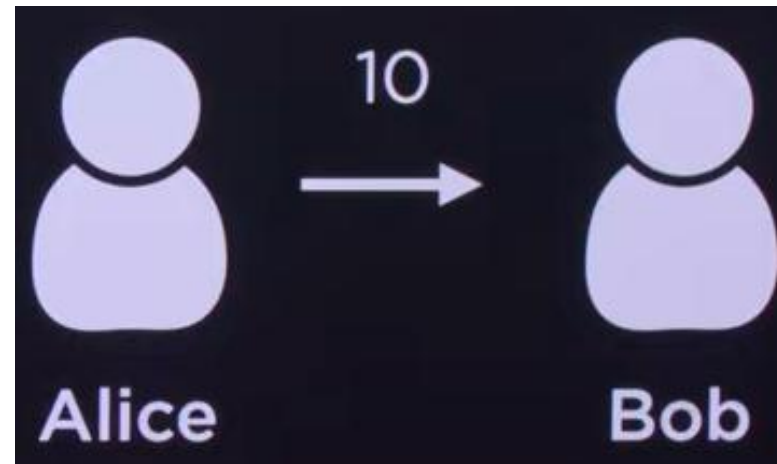
UPDATE accounts SET balance = balance + 10 WHERE id = 2;

UPDATE accounts SET balance = balance - 10 WHERE id = 1;

COMMIT;

再进行一笔事务

id	name	balance
1	Alice	-10
2	Bob	40
3	Charlie	30



事务 - 回滚

BEGIN TRANSACTION;

...

ROLLBACK;

```
BEGIN TRANSACTION;  
    UPDATE accounts SET balance = balance + 10 WHERE id = 2;  
    UPDATE accounts SET balance = balance - 10 WHERE id = 1;  
ROLLBACK;
```


Charlie Alice

[illegible]

竞争条件



Transfer \$30 to Alice	Transfer \$30 to Alice	Withdraw \$60
Charlie's balance: \$30		
	Charlie's balance: \$30	
+\$30 to Alice		
	+\$30 to Alice	
		Alice's balance: \$60
		Withdraw \$60
-\$30 from Charlie		
	-\$30 from Charlie	
	CHECK constraint failed	

竞争条件 - 重新组织



Transfer \$30 to Alice	Transfer \$30 to Alice	Withdraw \$60
Begin transaction		
Charlie's balance: \$30		
+\$30 to Alice		
-\$30 from Charlie		
Commit		
	Begin transaction	
	Charlie's balance: \$0	
	+\$30 to Alice	
	-\$30 from Charlie	
	Rollback	

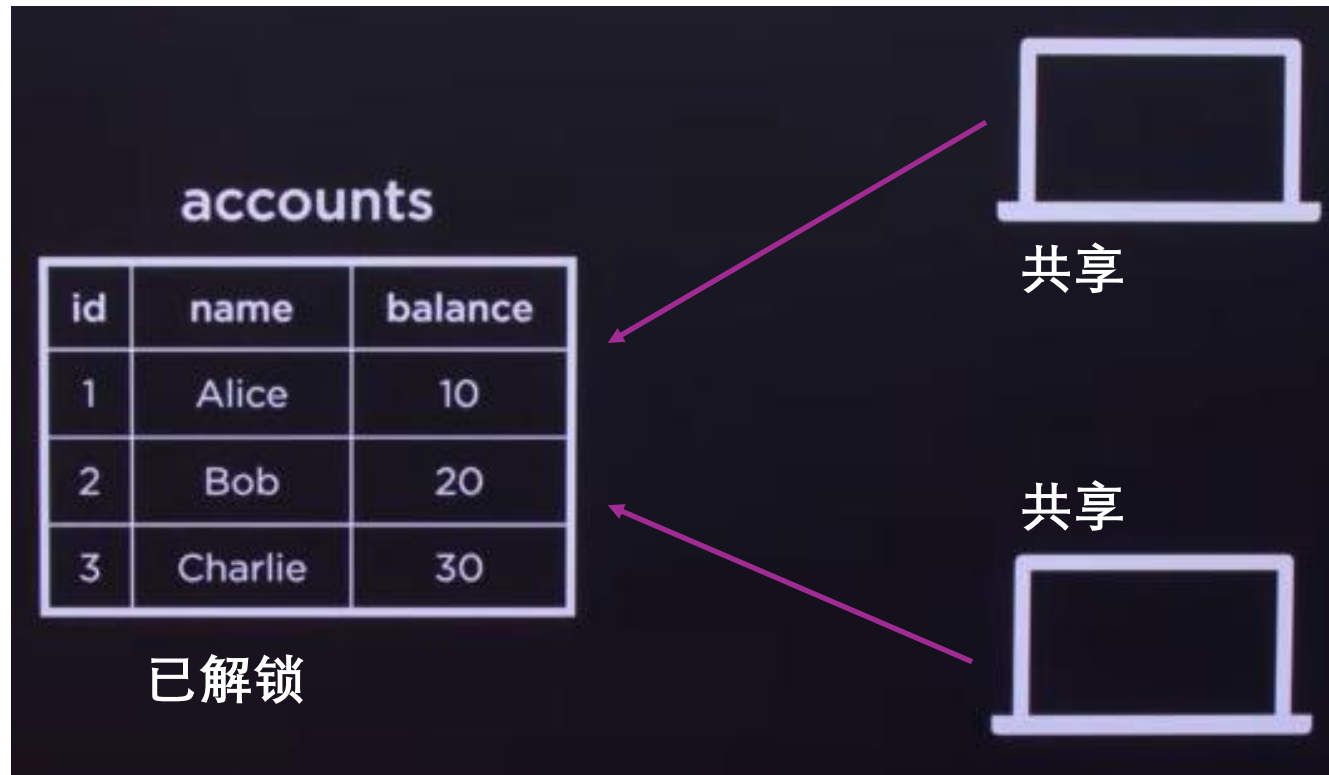
竞争条件



Transfer \$30 to Alice	Transfer \$30 to Alice	Withdraw \$60
		Begin transaction
		Alice's balance: \$60
		Withdraw \$60
		Commit

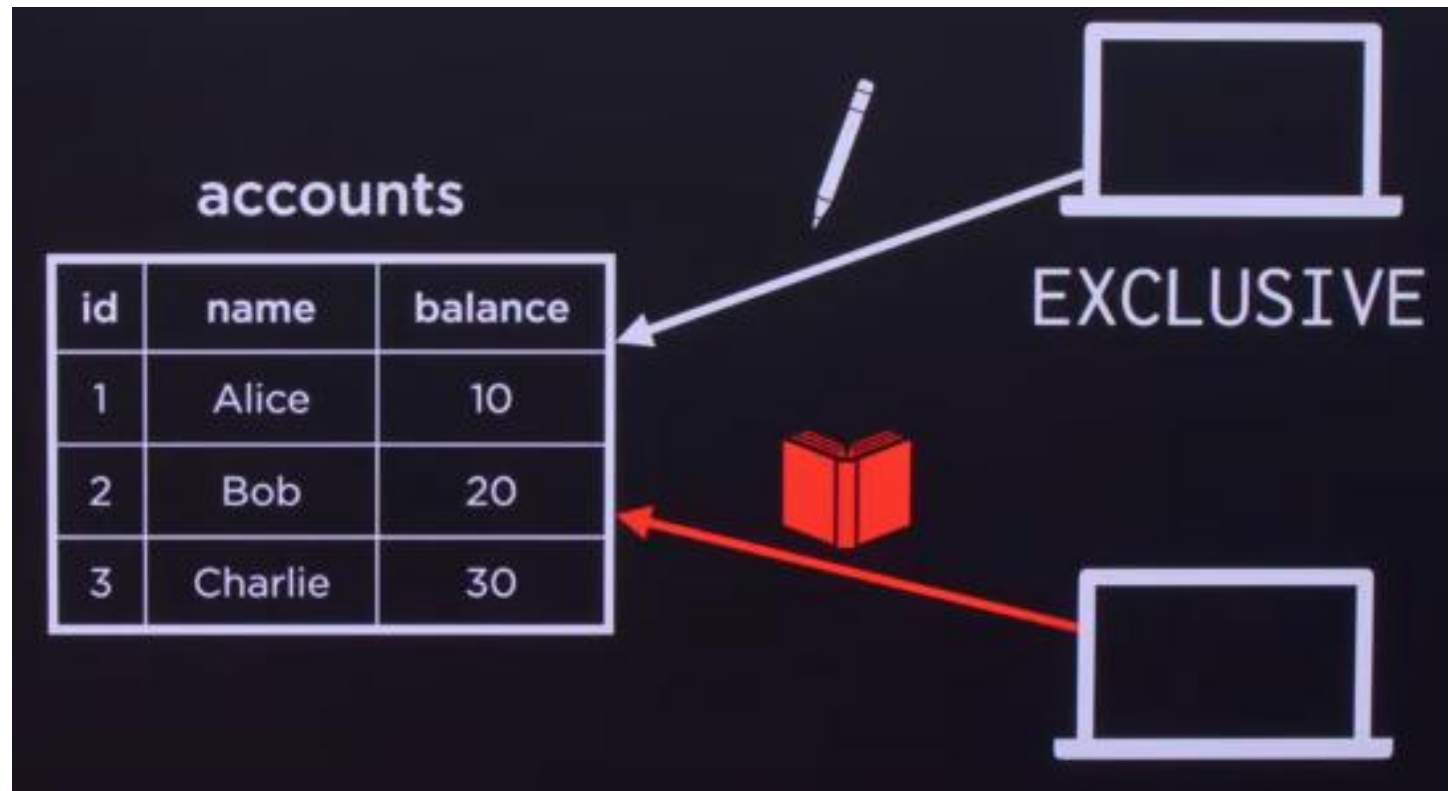
竞争条件 - 锁定

- 已解锁 UNLOCKED
- 共享 SHARED
- 独家的 EXCLUSIVE



竞争条件 - 独家锁定

BEGIN EXCLUSIVE TRANSACTION;



Database Systems

数据库系统

谢谢!

问题?

Instructor: Yiwen Shen (沈轶闻), Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr